

COMPUTER SCIENCE TRIPOS Part IB, Part II 50% – 2022 – Paper 7

10 Prolog (ijl20)

When answering this question ensure each relation has a comment giving a declarative reading of its behaviour. You should avoid unnecessary use of cut and not use extra-logical relations such as `findall`, `assertz` and `not (\+)`. Built-in library relations should not be assumed. The `notmember` relation given in the first part may be re-used if required.

lists, facts, rules,  
compound terms

- (a) Assume the built-in operator `\=` meaning *not unifiable with*, and a relation `notmember(+A,+L)` defined thus:

```
notmember(_, []).
notmember(A, [H|T]) :- A \= H, notmember(A, T).
```

Explain where *facts*, *rules*, *atoms*, *compound terms* have been used. Why does `notmember(A, [a,b,c])` fail? [2 marks]

---

*Answer:* `notmember(_, [])` is a *fact*. `notmember(A, [H|T]) :- ...` is a *rule*. `[]` is an *atom*. `[H|T]` is a *compound term* - 1 mark. `notmember(A, [a,b,c])` fails because `[a,b,c]` will not unify with `[]` in the first clause, and `A \= a` fails in the body of the second clause - 1 mark. *Alternative* acceptable answer would be any valid reference to the closed world assumption - 1 mark.

---

lists, relations,  
rules, facts

- (b) Write a `reverse(+A,?B)` relation suitable for *last call optimisation*. What makes it suitable for LCO? [3 marks]

---

*Answer:*

```
% reverse(+A,?B) succeeds if list B is the reversed list A
reverse(A,B) :- reverse_acc(A,[],B).
% reverse_acc(+A,+Acc,?B) succeeds if B is (the reverse of list A) appended
% to list Acc.
reverse_acc([],Acc,Acc).
reverse_acc([H|T],Acc,B) :- reverse_acc(T,[H|Acc],B).
```

Declarative comments for each relation 1 mark. Working 3-clause answer 1 mark. The relation is suitable for LCO because the last call of `reverse_acc` is determinate, i.e. there are no remaining choice points at the time of the call - 1 mark.

---

data  
representation

- (c) This small diagram represents our world map, with the arrows representing downhill lanes between places on the map.

```
house → lane
  ↓      ↓
field → forest
  ↓      ↓
lake → cave
```

Represent these downhill lanes with a `downhill(?A,?B)` relation. [2 marks]

---

*Answer:*

```
% downhill(?A,?B) succeeds if there is a downhill lane between A and B
```

```
downhill(house, lane).
downhill(house, field).
downhill(lane, forest).
downhill(field, forest).
downhill(field, lake).
downhill(forest, cave).
downhill(lake, cave).
```

Declarative comment 1 mark. All 7 correct `downhill` facts 1 mark.

---

recursion, graph  
search

- (d) Assuming `downhill(?A,?B)` is acyclic, define a relation `downhill_path(?A,?B)` which succeeds if place B can be reached from place A along downhill lanes. [2 marks]

---

*Answer:*

```
% downhill_path(?A,?B) succeeds if B can be reached downhill from A.
downhill_path(A,B) :- downhill(A,B).
downhill_path(A,B) :- downhill(A,C), downhill_path(C,B).
```

Succinct `downhill(?A,?B)` relation using 2 clauses 1 mark. Correct solution 1 mark.

---

relation

- (e) Define a relation `linked(?A,?B)` which succeeds if a lane *directly* connects places A and B downhill or the reverse, e.g. `:- linked(cave,forest)` should succeed. [2 marks]

---

*Answer:*

```
% linked(?A,?B) succeeds if there is a lane
% uphill or downhill between A and B.
linked(A,B) :- downhill(A,B).
linked(A,B) :- downhill(B,A).
```

Declarative comment 1 mark. Working 2-clause solution 1 mark.

---

relation,  
accumulator

- (f) Define a relation `linked_path(+A,+B,?Path)` which finds a linked path between places A and B, reporting the ordered list of places visited from A to B in the `Path` argument. [6 marks]

---

*Answer:*

```
% linked_path(+A,+B,?Path) succeeds if Path contains a sequence of places
% representing a path (uphill or down) between A and B.
```

```
linked_path(A,B,Path) :-
    linked_path_acc(A,B,[A],RPath),
    reverse(RPath, Path).
```

```
% linked_path_acc(+A,+B,+Path,?P) succeeds if P the reverse sequence of
% places from A to B excluding A, followed by Path.
```

```
linked_path_acc(A,A,Path,Path).
linked_path_acc(A,B,Path,P) :-
    linked(A,C),
    notmember(C,Path),
```

```
linked_path_acc(C,B,[C|Path],P).
```

Note `notmember/2` is likely to be required to ensure termination after backtracking. Declarative comments for each relation 2 marks. Correct implementation of `linked_path/3` given a subsidiary accumulator relation 1 mark. Correct implementation of the path accumulator relation 2 marks. Succinct solution in 3 clauses 1 mark.

---

relation,  
accumulator

- (g) Assume a relation `danger(?A,?D)` where D gives a numerical value for the danger at each place A, for example `:- danger(forest,X)` might succeed with `X=4`. Extend your `linked_path` relation so that it also returns the sum of the danger values along the path, i.e. `linked_path(+A,+B,?Path,?Danger)` [3 marks]
- 

*Answer:*

```
% danger(?A,?D) succeeds if D is the danger level at place A
danger(house, 1).
danger(lane, 2).
danger(field, 3).
danger(forest,4).
danger(lake, 5).
danger(cave, 6).

% linked_path(+A,+B,?Path,?Danger) succeeds if Path contains a sequence of
% places representing a path (uphill or down) between A and B, and Danger
% contains the sum of danger values for each place in Path.
linked_path(A,B,Path,Danger) :-
    linked_path_acc(A,B,[A],RPath,Danger),
    reverse(RPath, Path).

% linked_path_acc(+A,+B,+P,?Path,?D) succeeds if Path the reverse sequence of
% places from A to B excluding A, followed by P, and D is the sum of
% danger from A to B.
linked_path_acc(A,A,Path,Path,D) :- danger(A,D).
linked_path_acc(A,B,Path,P,D) :-
    linked(A,C),
    notmember(C,Path),
    danger(A,D1),
    linked_path_acc(C,B,[C|Path], P, D2),
    D is D1+D2.
```

Declarative comments 1 mark, correct sum for Danger 1 mark, simple extension to answer in (e) rather than a `sum(+Path,?D)` relation and a second pass through the path list 1 mark.

---